

Phase 7: Strict Binary Trees

0. Purpose of This Phase

Phase 7 teaches **strict binary trees**.

Before this phase, you learned ordinary binary trees. An ordinary binary tree allows each node to have:

- 0 children
- 1 child
- 2 children

A **strict binary tree** is more restricted.

The main beginner idea is:

A strict binary tree does not allow any node to have exactly one child.

In a strict binary tree, every node must have either:

0 children
or
2 children

No node is allowed to have exactly one child.

1. Current Progress Analysis

You have already completed the ordinary binary tree foundation from Phases 1 to 6.

Phase	What you learned	Why it matters in Phase 7
Phase 1	Tree vocabulary: root, child, leaf, internal node, degree, height	Strict trees depend heavily on node degree.
Phase 2	Binary tree formulas and degree 0, degree 1, and degree 2 nodes	Strict trees are based on allowing only degree 0 and degree 2 nodes.
Phase 3	Linked representation using <code>Node*</code> , <code>left</code> , <code>right</code> , and <code>nullptr</code>	Strict tree validation checks child pointers.
Phase 4	Recursive traversals	Phase 7 uses recursion to inspect every node.

Phase	What you learned	Why it matters in Phase 7
Phase 5	Iterative traversals and level order	You understand stack and queue traversal, but Phase 7 mainly uses recursion.
Phase 6	Count nodes, count degree-2 nodes, sum values, calculate height	Phase 7 reuses the same recursive idea: solve left subtree, solve right subtree, then combine.

So your current position is:

You already know how to build, traverse, and analyze binary trees. Now you are learning a special type of binary tree where every node must have 0 or 2 children.

2. What Phase 7 Covers

Phase 7 covers these main topics:

1. Meaning of a strict binary tree
2. Degree rule for strict binary trees
3. Valid and invalid strict binary tree examples
4. How to check if a binary tree is strict
5. Recursive strict tree validation
6. Dry run of strict tree validation
7. Strict binary tree height-node formulas
8. Internal and external node relationships
9. Complete beginner C++ program
10. Common beginner mistakes
11. Final memory tricks
12. Practice questions

Part A: Strict Binary Tree Concept

3. What Is a Strict Binary Tree?

A **strict binary tree** is a binary tree in which every node has either:

0 children
or
2 children

That means a node with exactly one child is not allowed.

Simple definition:

Strict binary tree = every node has 0 or 2 children

Another name for a strict binary tree is:

Proper binary tree

So:

Strict binary tree = Proper binary tree

Both names mean the same idea in this phase.

4. Ordinary Binary Tree vs Strict Binary Tree

An ordinary binary tree allows more freedom.

Type of tree	Children allowed for each node
Ordinary binary tree	0, 1, or 2 children
Strict binary tree	0 or 2 children only

Important difference:

A normal binary tree can have a node with one child. A strict binary tree cannot.

5. Degree Review

The **degree of a node** means the number of direct children that node has.

Degree	Meaning
Degree 0	Node has no children. It is a leaf.
Degree 1	Node has exactly one child.
Degree 2	Node has exactly two children.

In a strict binary tree:

Node degree	Allowed?	Reason
Degree 0	Yes	Leaf nodes are allowed.
Degree 1	No	A strict binary tree does not allow single-child nodes.
Degree 2	Yes	Internal nodes must have exactly two children.

Simple memory trick:

Strict tree allows degree 0 and degree 2 only.
Degree 1 is forbidden.

6. The Fastest Way to Check Strictness

The fastest rule is:

If you see a node with exactly one child, the tree is not strict.

That means:

- A leaf node is fine.
- A node with two children is fine.
- A node with only a left child is not fine.
- A node with only a right child is not fine.

Memory trick:

One lonely child breaks strictness.

Part B: Examples

7. Example of a Strict Binary Tree

Consider this tree:

```

      A
     /
    B  C
  
```



Now check each node.

Node	Children	Degree	Is it allowed in a strict tree?
A	B and C	2	Yes
B	D and E	2	Yes
C	none	0	Yes
D	none	0	Yes
E	none	0	Yes

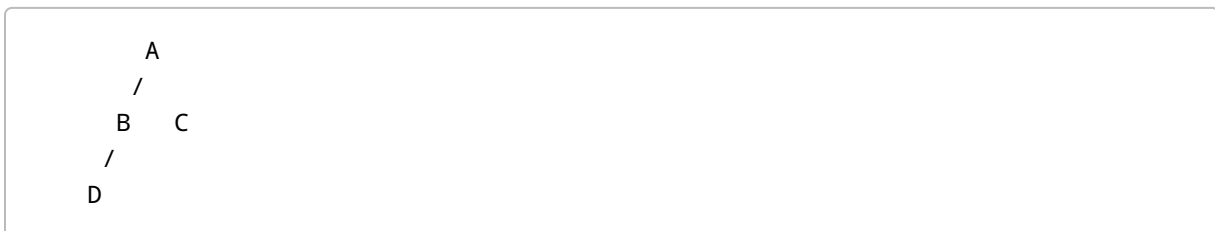
Every node has either 0 or 2 children.

So this tree is:

Strict

8. Example of a Tree That Is Not Strict

Consider this tree:



Now check each node.

Node	Children	Degree	Is it allowed in a strict tree?
A	B and C	2	Yes
B	D only	1	No
C	none	0	Yes
D	none	0	Yes

Node **B** has exactly one child.

So the whole tree is:

Not strict

Important idea:

Even if only one node breaks the rule, the entire tree is not strict.

9. Strict Tree with Only One Node

A single-node tree looks like this:

A

Node **A** has no children.

So:

Degree of A = 0

Degree 0 is allowed in a strict binary tree.

Therefore, a single-node tree is strict.

Important:

A root can also be a leaf.

10. Strict Tree with Root and Two Children

Consider this tree:

```
  A
 /
B  C
```

Check each node:

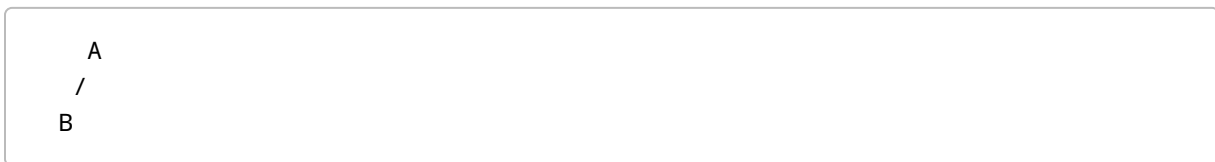
Node	Children	Degree
A	B and C	2
B	none	0
C	none	0

All nodes have degree 0 or degree 2.

So this tree is strict.

11. Not Strict Tree with Only Left Child

Consider this tree:



Node **A** has exactly one child.

So:

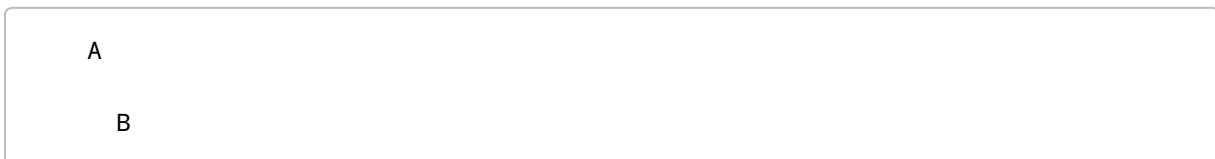
Degree of A = 1

Degree 1 is not allowed.

Therefore, this tree is not strict.

12. Not Strict Tree with Only Right Child

Consider this tree:



Node **A** has exactly one child.

So:

Degree of A = 1

Degree 1 is not allowed.

Therefore, this tree is not strict.

Part C: Validating a Strict Binary Tree

13. What Does Validation Mean?

To **validate** a strict binary tree means:

Check whether the tree follows the strict binary tree rule.

We want to write a function that answers this question:

Is this binary tree strict or not?

The function should return:

```
true  -> the tree is strict  
false -> the tree is not strict
```

In C++, the function can look like this:

```
bool isStrict(Node* root)
```

14. Recursive Thinking for Strict Validation

At each node, there are four possible cases.

Case 1: The node is empty

If the node is `nullptr`, there is no node here.

```
if (root == nullptr) return true;
```


An empty position does not break the strict rule.

So we return:

```
true
```

Case 2: The node is a leaf

A leaf has no left child and no right child.

```
if (root->left == nullptr && root->right == nullptr) return true;
```

A leaf has 0 children.

Degree 0 is allowed in a strict binary tree.

So we return:

```
true
```

Case 3: The node has two children

A node has two children when:

```
root->left != nullptr && root->right != nullptr
```

This node itself follows the strict rule.

But we still need to check the left subtree and right subtree.

So we return:

```
return isStrict(root->left) && isStrict(root->right);
```

This means:

The current node is okay.
Now check whether the left subtree is strict AND the right subtree is strict.

Both sides must be strict.

Case 4: The node has exactly one child

If the node is not empty, not a leaf, and not a two-child node, then it must have exactly one child.

That breaks the strict rule.

So we return:

```
return false;
```

15. Strict Validation Code

```
bool isStrict(Node* root) {  
    if (root == nullptr) {  
        return true;  
    }  
  
    if (root->left == nullptr && root->right == nullptr) {  
        return true;  
    }  
  
    if (root->left != nullptr && root->right != nullptr) {  
        return isStrict(root->left) && isStrict(root->right);  
    }  
  
    return false;  
}
```

16. Explanation of the Code

Empty tree check

```
if (root == nullptr) {  
    return true;  
}
```

If there is no node, there is no violation.

Leaf node check

```
if (root->left == nullptr && root->right == nullptr) {  
    return true;  
}
```

If the node has no children, it is a leaf.

A leaf is allowed in a strict binary tree.

Two-child node check

```
if (root->left != nullptr && root->right != nullptr) {  
    return isStrict(root->left) && isStrict(root->right);  
}
```

If the node has two children, the current node is valid.

But the function must still check both subtrees.

The `&&` means:

AND

So this line means:

Return true only if the left subtree is strict AND the right subtree is strict.

One-child node case

```
return false;
```

If none of the previous cases matched, the node must have exactly one child.

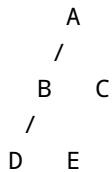
That is not allowed.

So the function returns false.

Part D: Dry Runs

17. Dry Run 1: Strict Tree

Use this tree:



Call:

```
isStrict(A)
```

Step-by-step:

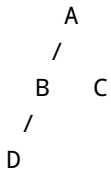
Step	Function call	What happens?	Result
1	<code>isStrict(A)</code>	A has two children, so check B and C	Continue
2	<code>isStrict(B)</code>	B has two children, so check D and E	Continue
3	<code>isStrict(D)</code>	D is a leaf	true
4	<code>isStrict(E)</code>	E is a leaf	true
5	<code>isStrict(B)</code>	D and E are both strict	true
6	<code>isStrict(C)</code>	C is a leaf	true
7	<code>isStrict(A)</code>	B and C are both strict	true

Final answer:

The tree is strict.

18. Dry Run 2: Not Strict Tree

Use this tree:



Call:

isStrict(A)

Step-by-step:

Step	Function call	What happens?	Result
1	isStrict(A)	A has two children, so check B and C	Continue
2	isStrict(B)	B has only left child D	false
3	isStrict(A)	Left subtree is not strict	false

Final answer:

The tree is not strict.

Why?

Node B has exactly one child.

19. Dry Run 3: Single Node Tree

Use this tree:

A

Call:

isStrict(A)

Node **A** has no left child and no right child.

So **A** is a leaf.

A leaf has degree 0.

Degree 0 is allowed.

Final answer:

The tree is strict.

Part E: Strict Binary Tree Formulas

20. Why Strict Tree Formulas Are Different

Ordinary binary trees can grow like a chain.

Example:

A

B

C

This is valid as an ordinary binary tree.

But it is not strict because each internal node has only one child.

A strict binary tree cannot grow using single-child chains.

That is why strict binary tree formulas are different from ordinary binary tree formulas.

21. Height Convention

In this curriculum, height counts edges.

A single root has height 0.

Example:

A

Height:

0

Example:

A
/
B C

Longest path has 1 edge.

Height:

1

22. Minimum Nodes for Height H in a Strict Binary Tree

For an ordinary binary tree, the minimum number of nodes for height H is:

$H + 1$

That works because ordinary binary trees can be chains.

But strict binary trees cannot be chains.

For a strict binary tree:

Minimum nodes for height $H = 2H + 1$

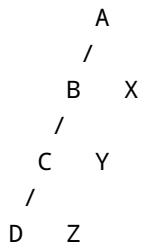
23. Why Minimum Nodes = $2H + 1$

To create height H , we need a path of length H edges.

But because the tree is strict, every internal node on that path must have a second child.

So each level on the path needs extra leaf nodes to keep the tree strict.

Example for height 3:



Longest path:

A -> B -> C -> D

This path has 3 edges, so height is 3.

Count nodes:

A, B, C, D, X, Y, Z = 7 nodes

Formula:

$$2H + 1 = 2(3) + 1 = 7$$

24. Minimum Nodes Formula Examples

Height H	Minimum nodes	$2H + 1$
0		1
1		3
2		5
3		7
4		9

Simple memory trick:

Strict minimum nodes grow by 2 each time height increases by 1.

25. Maximum Nodes for Height H in a Strict Binary Tree

To get the maximum number of nodes, fill every level completely.

The formula is:

Maximum nodes for height H = $2^{(H + 1)} - 1$

This is the same maximum formula used for ordinary binary trees.

26. Maximum Nodes Formula Example

If:

H = 3

Then:

Maximum nodes = $2^{(H + 1)} - 1$
= $2^{(3 + 1)} - 1$
= $2^4 - 1$

$$= 16 - 1$$

$$= 15$$

So a strict binary tree of height 3 can have at most 15 nodes.

27. Maximum Nodes Formula Table

Height H	Maximum nodes $2^{(H + 1)} - 1$
0	1
1	3
2	7
3	15
4	31

28. Node Range for Height H

For a strict binary tree with height H:

$$\text{Minimum nodes} = 2H + 1$$

$$\text{Maximum nodes} = 2^{(H + 1)} - 1$$

So:

$$2H + 1 \leq N \leq 2^{(H + 1)} - 1$$

Example:

If:

$$H = 3$$

Then:

Minimum nodes = $2(3) + 1 = 7$
Maximum nodes = $2^{(3 + 1)} - 1 = 15$

So the number of nodes must be between:

7 and 15

29. Minimum Height for N Nodes

To make height as small as possible, pack the nodes as tightly as possible.

Formula:

Minimum height for N nodes = $\text{ceil}(\log_2(N + 1)) - 1$

This is the same idea as ordinary binary trees because the shortest height happens when the tree is packed.

30. Maximum Height for N Nodes in a Strict Binary Tree

To make height as large as possible in a strict binary tree, use the minimum-node shape.

We know:

$$N = 2H + 1$$

Solve for **H**:

$$\begin{aligned} N - 1 &= 2H \\ H &= (N - 1) / 2 \end{aligned}$$

So:

Maximum height for N nodes = $(N - 1) / 2$

This works because a strict binary tree has an odd number of nodes.

31. Formula Summary

Question	Formula
Minimum nodes for height H	$2H + 1$
Maximum nodes for height H	$2^{(H + 1)} - 1$
Minimum height for N nodes	$\text{ceil}(\log_2(N + 1)) - 1$
Maximum height for N nodes	$(N - 1) / 2$

Part F: Internal and External Nodes

32. Internal Node in a Strict Binary Tree

In a strict binary tree, an **internal node** is a node with two children.

Why?

Because strict trees do not allow degree-1 nodes.

So every non-leaf node must have exactly two children.

Internal node = node with 2 children

33. External Node in a Strict Binary Tree

An **external node** is a leaf node.

A leaf has no children.

External node = leaf node = node with 0 children

34. Internal and External Node Relationship

Let:

```
i = number of internal nodes
e = number of external nodes / leaves
n = total number of nodes
```

In a strict binary tree:

```
e = i + 1
```

This means:

```
Number of leaves = Number of internal nodes + 1
```

35. Total Nodes Formula

Since total nodes are internal nodes plus external nodes:

```
n = i + e
```

But:

```
e = i + 1
```

So:

```
n = i + (i + 1)
n = 2i + 1
```

Therefore:

```
Total nodes = 2 * internal nodes + 1
```

36. Why Strict Binary Trees Always Have Odd Nodes

The total node formula is:

$$n = 2i + 1$$

Any number in the form $2i + 1$ is odd.

Examples:

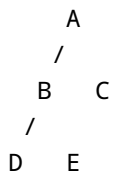
Internal nodes i	Total nodes $2i + 1$
0	1
1	3
2	5
3	7
4	9
5	11

So:

A strict binary tree always has an odd number of nodes.

37. Example: Internal and External Nodes

Consider this strict binary tree:



Internal nodes:

A, B

So:

$$i = 2$$

External nodes / leaves:

C, D, E

So:

$$e = 3$$

Check the formula:

$$\begin{aligned} e &= i + 1 \\ 3 &= 2 + 1 \end{aligned}$$

Correct.

Total nodes:

$$n = 5$$

Check total formula:

$$\begin{aligned} n &= 2i + 1 \\ n &= 2(2) + 1 \\ n &= 5 \end{aligned}$$

Correct.

38. Example Calculation

Suppose a strict binary tree has:

$i = 5$ internal nodes

Find the number of leaves.

Formula:

$$e = i + 1$$

Substitute:

$$\begin{aligned} e &= 5 + 1 \\ &= 6 \end{aligned}$$

So the tree has:

6 leaves

Find total nodes.

Formula:

$$n = 2i + 1$$

Substitute:

$$\begin{aligned} n &= 2(5) + 1 \\ &= 10 + 1 \\ &= 11 \end{aligned}$$

So the tree has:

11 total nodes

Part G: Complete Beginner C++ Program

39. Complete Program

```
#include <iostream>
using namespace std;
```



```

struct Node {
    char data;
    Node* left;
    Node* right;

    Node(char value) {
        data = value;
        left = nullptr;
        right = nullptr;
    }
};

bool isStrict(Node* root) {
    if (root == nullptr) {
        return true;
    }

    // Leaf node: 0 children
    if (root->left == nullptr && root->right == nullptr) {
        return true;
    }

    // Internal node with 2 children
    if (root->left != nullptr && root->right != nullptr) {
        return isStrict(root->left) && isStrict(root->right);
    }

    // Node with exactly 1 child
    return false;
}

int minNodesStrict(int height) {
    return 2 * height + 1;
}

int maxNodesStrict(int height) {
    int result = 1;
    for (int i = 0; i < height + 1; i++) {
        result *= 2;
    }
    return result - 1;
}

int leavesFromInternal(int internalNodes) {
    return internalNodes + 1;
}

int totalNodesFromInternal(int internalNodes) {

```

```

        return 2 * internalNodes + 1;
    }

    int main() {
        /*
            A
           /
          B  C
         /
        D  E
        */

        Node* root = new Node('A');
        root->left = new Node('B');
        root->right = new Node('C');
        root->left->left = new Node('D');
        root->left->right = new Node('E');

        if (isStrict(root)) {
            cout << "The tree is strict." << endl;
        } else {
            cout << "The tree is not strict." << endl;
        }

        int h = 3;
        cout << "Minimum nodes for strict tree of height " << h << ": "
              << minNodesStrict(h) << endl;

        cout << "Maximum nodes for strict tree of height " << h << ": "
              << maxNodesStrict(h) << endl;

        int internal = 5;
        cout << "Leaves for " << internal << " internal nodes: "
              << leavesFromInternal(internal) << endl;

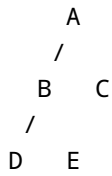
        cout << "Total nodes for " << internal << " internal nodes: "
              << totalNodesFromInternal(internal) << endl;

        return 0;
    }

```

40. Tree Created by the Program

The program creates this tree:



Node facts:

Node	Children	Degree
A	B and C	2
B	D and E	2
C	none	0
D	none	0
E	none	0

Every node has degree 0 or degree 2.

So the tree is strict.

41. Expected Output

```

The tree is strict.
Minimum nodes for strict tree of height 3: 7
Maximum nodes for strict tree of height 3: 15
Leaves for 5 internal nodes: 6
Total nodes for 5 internal nodes: 11

```

42. Explanation of the Program

Node structure

```

struct Node {
    char data;
    Node* left;
    Node* right;
};

```

Each node stores:

1. Its data
 2. Address of left child
 3. Address of right child
-

Constructor

```
Node(char value) {  
    data = value;  
    left = nullptr;  
    right = nullptr;  
}
```

This creates a node and sets both child pointers to `nullptr`.

This means the node starts with no children.

Strict check function

```
bool isStrict(Node* root)
```

This function checks whether the tree is strict.

It returns:

```
true  if the tree is strict  
false if the tree is not strict
```

Minimum node function

```
int minNodesStrict(int height) {  
    return 2 * height + 1;  
}
```

This returns the minimum number of nodes possible for a strict binary tree with the given height.

Maximum node function

```
int maxNodesStrict(int height) {  
    int result = 1;  
    for (int i = 0; i < height + 1; i++) {  
        result *= 2;  
    }  
    return result - 1;  
}
```

This calculates:

```
2^(height + 1) - 1
```

Leaves from internal nodes

```
int leavesFromInternal(int internalNodes) {  
    return internalNodes + 1;  
}
```

This uses the formula:

```
leaves = internal nodes + 1
```

Total nodes from internal nodes

```
int totalNodesFromInternal(int internalNodes) {  
    return 2 * internalNodes + 1;  
}
```

This uses the formula:

```
total nodes = 2 * internal nodes + 1
```

Part H: Complexity Analysis

43. Time Complexity of `isStrict`

The `isStrict` function may visit every node once.

So if the tree has `N` nodes:

Time complexity = $O(N)$

Why?

Because in the worst case, the function must check every node to be sure the tree is strict.

44. Space Complexity of `isStrict`

The function is recursive, so it uses the system stack.

The maximum stack depth depends on the height of the tree.

So:

Space complexity = $O(H)$

Here:

H = height of the tree

For a balanced tree, the height is smaller.

For a tall tree, the height is larger.

45. Complexity Summary

Function	Time Complexity	Extra Space
<code>isStrict</code>	$O(N)$	$O(H)$
<code>minNodesStrict</code>	$O(1)$	$O(1)$

Function	Time Complexity	Extra Space
<code>maxNodesStrict</code>	$O(H)$ in the loop version	$O(1)$
<code>leavesFromInternal</code>	$O(1)$	$O(1)$
<code>totalNodesFromInternal</code>	$O(1)$	$O(1)$

Note:

`maxNodesStrict` can also be written using a power function, but the loop version is easier for beginners to understand.

Part I: Common Beginner Mistakes

46. Mistake 1: Thinking Every Binary Tree Is Strict

Wrong idea:

Every binary tree is strict.

Correct idea:

Only binary trees with no degree-1 nodes are strict.

Example:

```
A
 /
B
```

This is a binary tree, but it is not strict.

Why?

Because `A` has exactly one child.

47. Mistake 2: Thinking a Leaf Breaks Strictness

Wrong idea:

A leaf has no children, so it is not strict.

Correct idea:

A leaf is allowed because it has 0 children.

Strict means:

0 or 2 children

So 0 children is valid.

48. Mistake 3: Forgetting That Degree 1 Is Forbidden

A node with one child is not allowed.

Both of these break strictness:

A
/
B

and:

A

B

In both cases, node **A** has exactly one child.

49. Mistake 4: Using Ordinary Minimum Node Formula

For ordinary binary trees:

minimum nodes = $H + 1$

But for strict binary trees:

minimum nodes = $2H + 1$

Why?

Because strict trees cannot use single-child chains.

50. Mistake 5: Forgetting to Check Both Subtrees

Wrong thinking:

If the root has 0 or 2 children, the tree is strict.

Correct thinking:

The root must be valid, and every node under it must also be valid.

That is why recursion is needed.

51. Mistake 6: Confusing Strict with Complete or Full

Strict, complete, and full trees are related terms, but they are not the same idea.

In this phase:

Strict controls the number of children per node.

Strict rule:

Every node has 0 or 2 children.

Complete and full tree ideas are discussed more deeply in Phase 8.

Part J: Final Memory Tricks

52. Main Strict Rule

Strict binary tree = every node has 0 or 2 children.

53. Bad Node Rule

A node with exactly one child breaks strictness.

54. Validation Rule

Empty node -> true
Leaf node -> true
Two-child node -> check left and right subtrees
One-child node -> false

55. Formula Memory Table

Idea	Formula
Minimum nodes for height H	$2H + 1$
Maximum nodes for height H	$2^{(H + 1)} - 1$
Minimum height for N nodes	$\text{ceil}(\log_2(N + 1)) - 1$
Maximum height for N nodes	$(N - 1) / 2$
External nodes from internal nodes	$e = i + 1$
Total nodes from internal nodes	$n = 2i + 1$

56. Shortest Summary

A strict binary tree is a binary tree where every node has either no children or exactly two children.

Degree 0 is allowed.
Degree 1 is not allowed.
Degree 2 is allowed.

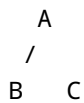
If any node has exactly one child, the whole tree is not strict.

Strict tree formulas:
minimum nodes = $2H + 1$
maximum nodes = $2^{(H + 1)} - 1$
leaves = internal nodes + 1
total nodes = $2 * \text{internal nodes} + 1$

Part K: Practice Questions

57. Practice Question 1

Is this tree strict?



Answer:

Yes, it is strict.

Why?

- A has 2 children.
- B has 0 children.
- C has 0 children.

All nodes have 0 or 2 children.

58. Practice Question 2

Is this tree strict?

```
  A
 /
B
```

Answer:

No, it is not strict.

Why?

Node **A** has exactly one child.

59. Practice Question 3

A strict binary tree has height 4.

Find the minimum number of nodes.

Formula:

$\text{minimum nodes} = 2H + 1$

Substitute:

$$\begin{aligned}\text{minimum nodes} &= 2(4) + 1 \\ &= 8 + 1 \\ &= 9\end{aligned}$$

Answer:

9 nodes

60. Practice Question 4

A strict binary tree has height 4.

Find the maximum number of nodes.

Formula:

$$\text{maximum nodes} = 2^{(H + 1)} - 1$$

Substitute:

$$\begin{aligned}\text{maximum nodes} &= 2^{(4 + 1)} - 1 \\ &= 2^5 - 1 \\ &= 32 - 1 \\ &= 31\end{aligned}$$

Answer:

31 nodes

61. Practice Question 5

A strict binary tree has 6 internal nodes.

Find the number of leaves.

Formula:

$$\text{leaves} = \text{internal nodes} + 1$$

Substitute:

$$\begin{aligned}\text{leaves} &= 6 + 1 \\ &= 7\end{aligned}$$

Answer:

7 leaves

62. Practice Question 6

A strict binary tree has 6 internal nodes.

Find the total number of nodes.

Formula:

$$\text{total nodes} = 2 * \text{internal nodes} + 1$$

Substitute:

$$\begin{aligned}\text{total nodes} &= 2(6) + 1 \\ &= 12 + 1 \\ &= 13\end{aligned}$$

Answer:

13 total nodes

63. Practice Question 7

Can a strict binary tree have 10 total nodes?

Answer:

No.

Why?

A strict binary tree always has an odd number of nodes.

10 is even.

So a strict binary tree cannot have exactly 10 nodes.

64. Practice Question 8

Can a strict binary tree have 11 total nodes?

Answer:

Yes .

Why?

11 is odd.

Also:

$$n = 2i + 1$$

If:

$$11 = 2i + 1$$

Then:

$$\begin{aligned} 10 &= 2i \\ i &= 5 \end{aligned}$$

So a strict binary tree with 11 nodes has 5 internal nodes and 6 leaves.

65. Phase 7 Conclusion

In Phase 7, you learned strict binary trees.

The most important rule is:

Every node must have either 0 or 2 children.

A node with exactly one child is not allowed.

You also learned how to:

- Identify strict and non-strict trees
- Validate a strict tree using recursion
- Use strict binary tree formulas
- Find leaves from internal nodes
- Find total nodes from internal nodes
- Understand why strict trees always have an odd number of nodes

You are now ready for Phase 8, where strict binary trees are compared with full and complete binary trees.